1. Which of the following built-in Python sequences is immutable?

(A) List
(B) Tuple
(C) Dictionary
(D) Set

2. What is the return value of 'Python'[1:4]?

(A) 'Pyth'
(B) 'yth'
(C) 'ytho'
(D) 'Pyt'

3. Which method is used to remove all whitespace from both ends of a string?

(A) strip()
(B) trim()
(C) clean()
(D) split()

4. How is a single-element tuple correctly defined in Python?

(A) t = (42)
(B) t = (42,)
(C) t = tuple(42)
(D) t = [42]

5. What does the list method lst.append(5) return?

(A) The modified list
(B) None
(C) Integer 5
(D) Length of lst

6. Which list method adds all elements of an iterable to the end of the list?

(A) append()
(B) insert()
(C) extend()
(D) add()

7. What is the result of len({'a': 1, 'b': 2, 'a': 3})?

(A) 3
(B) 2
(C) 1
(D) Error: Duplicate Key

8. Which dictionary method safely retrieves a value without raising a KeyError?

(A) fetch()
(B) get()
(C) find()
(D) lookup()

9. What is the index of the first element in any Python sequence?

(A) 1
(B) 0
(C) -1
(D) Depends on type

10. What does the index -1 represent in a string or list?

(A) First element
(B) Second element
(C) Last element
(D) Invalid index

11. Which built-in function returns an iterable of (index, item) pairs?

(A) zip()
(B) range()
(C) enumerate()
(D) map()

12. What is the result of 'a' + 'b' in Python?

(A) 'ab'
(B) 'a b'
(C) Error
(D) ['a', 'b']

13. What does 'hello' * 3 produce?

(A) 'hellohellohello'
(B) ['hello', 'hello', 'hello']
(C) 'hello 3'
(D) TypeError

14. Which method joins elements of a list of strings with a specified delimiter?

(A) split()
(B) concat()
(C) join()
(D) merge()

15. What will 'ECE,CSE,ME'.split(',') evaluate to?

(A) ('ECE', 'CSE', 'ME')
(B) ['ECE', 'CSE', 'ME']
(C) {'ECE', 'CSE', 'ME'}
(D) 'ECE CSE ME'

16. Which method removes and returns the last element of a list?

(A) pop()
(B) remove()
(C) discard()
(D) delete()

17. What does lst.remove(x) do if element x is not present in the list?

(A) Returns None
(B) Returns False
(C) Raises ValueError
(D) Raises IndexError

18. Which operator checks if a key exists in a dictionary?

(A) has_key
(B) exists
(C) in
(D) contains

19. Which dictionary method returns a dynamic view of all key-value pairs as 2-tuples?

(A) pairs()
(B) items()
(C) entries()
(D) all()

20. What is the type of {1, 2, 3} in Python?

(A) dict
(B) set
(C) tuple
(D) list

21. What is the type of {} in Python?

(A) empty set
(B) empty dict
(C) empty tuple
(D) syntax error

22. How do you instantiate an empty set in Python?

(A) {}
(B) set()
(C) []
(D) set([])

23. Which set method computes elements present in set A but not in set B?

(A) intersection()
(B) difference()
(C) union()
(D) symmetric_difference()

24. What is the time complexity of checking membership ('x in s') in a hash set on average?

(A) O(1)
(B) O(log n)
(C) O(n)
(D) O(n log n)

25. What happens when executing s = 'hello'; s[0] = 'H'?

(A) s becomes 'Hello'
(B) TypeError: 'str' object does not support item assignment
(C) ValueError
(D) None

26. Which string method checks if all characters are decimal digits?

(A) isnumeric()
(B) isdigit()
(C) isdecimal()
(D) All of the above

27. What does 'Python'.find('th') return?

(A) 2
(B) 3
(C) True
(D) 1

28. What does 'Python'.find('xyz') return when the substring is absent?

(A) Raises ValueError
(B) None
(C) -1
(D) False

29. What does 'Python'.index('xyz') do when the substring is not found?

(A) Returns -1
(B) Returns None
(C) Raises ValueError
(D) Raises IndexError

30. Which string method converts the first character to uppercase and the rest to lowercase?

(A) upper()
(B) title()
(C) capitalize()
(D) swapcase()

31. What does list(range(1, 6)) return?

(A) [1, 2, 3, 4, 5, 6]
(B) [1, 2, 3, 4, 5]
(C) [0, 1, 2, 3, 4, 5]
(D) [2, 3, 4, 5]

32. What does lst[::-1] do to a list lst?

(A) Clears the list
(B) Reverses the list non-destructively
(C) Reverses the list in-place
(D) Returns an empty list

33. Which method sorts a list in-place and returns None?

(A) sorted()
(B) sort()
(C) order()
(D) arrange()

34. What does the built-in function sorted([3, 1, 2]) return?

(A) None
(B) [1, 2, 3]
(C) [3, 2, 1]
(D) Generator

35. Can a list be used as a key in a Python dictionary?

(A) Yes, always
(B) No, because lists are mutable and unhashable
(C) Only if length < 10
(D) Only if elements are ints

36. Can a tuple containing an inner list be used as a dictionary key?

(A) Yes, tuples are immutable
(B) No, because its contents are not hashable
(C) Yes, if cast to str
(D) Only in Python 3.12+

37. Which method empties all elements from a dictionary?

(A) delete()
(B) purge()
(C) clear()
(D) reset()

38. What is the output of d = {'a': 1}; d.pop('a')?

(A) 1
(B) {'a': 1}
(C) None
(D) KeyError

39. What is the result of (1, 2) + (3, 4)?

(A) (4, 6)
(B) (1, 2, 3, 4)
(C) [1, 2, 3, 4]
(D) TypeError

40. What does max(['apple', 'banana', 'cherry']) return?

(A) 'apple'
(B) 'banana'
(C) 'cherry'
(D) Length 6

41. What is the output of [x**2 for x in range(5) if x % 2 == 0]?

(A) [0, 1, 4, 9, 16]
(B) [0, 4, 16]
(C) [1, 9]
(D) [4, 16]

42. What is the output of {x: x.upper() for x in ['a', 'b']}?

(A) ['A', 'B']
(B) {'a': 'A', 'b': 'B'}
(C) {'A': 'a', 'B': 'b'}
(D) ('a', 'A')

43. What does `a, *b, c = [1, 2, 3, 4, 5]` assign to `b`?

(A) [1, 2, 3]
(B) [2, 3, 4]
(C) [3, 4, 5]
(D) (2, 3, 4)

44. What does the expression `dict(**d1, **d2)` do when d1 and d2 share keys?

(A) Raises KeyConflictError
(B) Values from d1 overwrite d2
(C) Values from d2 overwrite d1
(D) Merges values into lists

45. What operator was introduced in Python 3.9 for merging two dictionaries `d1 | d2`?

(A) Dictionary union operator
(B) Bitwise OR
(C) Pipe concatenation
(D) Bitwise XOR

46. What is the result of `list(zip(['a', 'b'], [1, 2, 3]))`?

(A) [('a', 1), ('b', 2), (None, 3)]
(B) [('a', 1), ('b', 2)]
(C) Raises ValueError
(D) [('a', 1, 2, 3)]

47. Which module provides `zip_longest` that fills shorter iterables with a fill value?

(A) collections
(B) itertools
(C) functools
(D) operator

48. What does `copy.deepcopy()` do differently compared to `copy.copy()`?

(A) Only copies primitive attributes
(B) Recursively copies all nested mutable objects
(C) Copies references only
(D) Runs faster than shallow copy

49. Consider `a = [[0]] * 3; a[0][0] = 5`. What is `a`?

(A) [[5], [0], [0]]
(B) [[5], [5], [5]]
(C) [[0], [0], [5]]
(D) [[5], [0]]

50. What is the correct way to initialize an independent 3x3 matrix in Python?

(A) [[0]*3]*3
(B) [[0 for _ in range(3)] for _ in range(3)]
(C) [[0]*3 for _ in 3]
(D) matrix(3, 3)

51. What does `str.split()` without arguments do when encountering multiple consecutive spaces?

(A) Produces empty strings in output
(B) Groups consecutive whitespace and discards leading/trailing
(C) Raises ValueError
(D) Splits only on single space

52. What does `str.partition(sep)` return?

(A) A list of 2 parts
(B) A 3-tuple (head, sep, tail)
(C) A set of parts
(D) A generator

53. Which string method maps characters according to a translation table created with `maketrans`?

(A) replace()
(B) translate()
(C) encode()
(D) format()

54. What does `collections.defaultdict(list)` do when an unseen key is accessed?

(A) Raises KeyError
(B) Initializes the key with an empty list
(C) Returns None
(D) Creates an empty tuple

55. Which class from `collections` counts frequency of hashable elements?

(A) FrequencyDict
(B) ElementCounter
(C) Counter
(D) Histogram

56. What does `collections.Counter('banana').most_common(1)` return?

(A) [('a', 3)]
(B) ('a', 3)
(C) {'a': 3}
(D) 3

57. What is the double-ended queue data structure in Python standard library?

- **(A)** queue.Queue
- **(B)** collections.deque
- **(C)** list.dequeue
- **(D)** sys.deque

58. What is the time complexity of `appendleft()` and `popleft()` on a `collections.deque`?

- **(A)** O(n)
- **(B)** O(log n)
- **(C)** O(1)
- **(D)** O(n^2)

59. What happens if you insert an element at index 0 of a standard Python list of length n?

- **(A)** O(1) time complexity
- **(B)** O(n) time complexity as all elements shift
- **(C)** O(log n) time complexity
- **(D)** Memory corruption

60. Which method returns a new frozenset containing elements common to two sets?

- **(A)** frozenset.union()
- **(B)** frozenset.intersection()
- **(C)** frozenset.add()
- **(D)** frozenset.difference()

61. Can a `frozenset` be used as a key in a Python dictionary?

- **(A)** No, sets cannot be hashed
- **(B)** Yes, because frozensets are immutable and hashable
- **(C)** Only if it contains numbers
- **(D)** Only in Python 2

62. What is the output of `sorted(['10', '2', '1'], key=int)`?

- **(A)** ['1', '10', '2']
- **(B)** ['1', '2', '10']
- **(C)** ['10', '2', '1']
- **(D)** [1, 2, 10]

63. What does `any([0, False, '', 42])` return?

- **(A)** False
- **(B)** True
- **(C)** 42
- **(D)** TypeError

64. What does `all([1, 'hello', [0], True])` return?

- **(A)** True
- **(B)** False
- **(C)** 1
- **(D)** None

65. What is the output of `list(filter(None, [0, 1, False, 2, '', 3]))`?

- **(A)** [0, False, '']
- **(B)** [1, 2, 3]
- **(C)** [0, 1, 2, 3]
- **(D)** TypeError

66. What does `'Signal {0:08b}'.format(5)` produce?

- **(A)** 'Signal 00000101'
- **(B)** 'Signal 101'
- **(C)** 'Signal 00000005'
- **(D)** 'Signal 5'

67. What does `f'{123.4567:.2f}'` produce?

- **(A)** '123.45'
- **(B)** '123.46'
- **(C)** '123.4'
- **(D)** '123.5'

68. How can you unpack a dictionary into keyword arguments of a function?

- **(A)** `func(*d)`
- **(B)** `func(**d)`
- **(C)** `func(&d)`
- **(D)** `func(d...)`

69. What is the result of `set([1, 2, 3]) ^ set([2, 3, 4])`?

- **(A)** {1, 4}
- **(B)** {2, 3}
- **(C)** {1, 2, 3, 4}
- **(D)** set()

70. What does `dict.setdefault(key, default)` return if key is ALREADY present?

- **(A)** The default value
- **(B)** The existing value for that key
- **(C)** None
- **(D)** KeyError

71. What does `dict.setdefault(key, default)` do if key is NOT present?

- **(A)** Sets key to default and returns default
- **(B)** Raises KeyError
- **(C)** Returns None without updating
- **(D)** Logs a warning

72. What is the output of `[i for sub in [[1, 2], [3, 4]] for i in sub]`?

- **(A)** [[1, 2], [3, 4]]
- **(B)** [1, 2, 3, 4]
- **(C)** [[1, 3], [2, 4]]
- **(D)** [4, 3, 2, 1]

73. Which string method returns True if all characters in the string are ASCII characters?

- **(A)** isascii()
- **(B)** isunicode()
- **(C)** isprintable()
- **(D)** isstandard()

74. What does `bytes([65, 66, 67]).decode('utf-8')` produce?

- **(A)** '[65, 66, 67]'
- **(B)** 'ABC'
- **(C)** 'abc'
- **(D)** TypeError

75. What is the mutable counterpart of the immutable `bytes` type?

- **(A)** bytearray
- **(B)** buffer
- **(C)** memoryview
- **(D)** chararray

76. How does CPython allocate capacity for dynamically resizing lists during `append()`?

- **(A)** Doubles size on each append
- **(B)** Over-allocates proportionally: 0, 4, 8, 16, 24, 32, 40...
- **(C)** Allocates exactly 1 item at a time
- **(D)** Allocates powers of 10

77. Why does `sys.getsizeof([])` return non-zero (typically 56 to 64 bytes) on a 64-bit machine?

- **(A)** Stores garbage data
- **(B)** Accounted for PyObject header, size, array pointer, and allocated capacity
- **(C)** Python memory leak
- **(D)** Reserved for OS heap header only

78. What collision resolution strategy does CPython's hash table use for dictionaries?

- **(A)** Separate chaining with linked lists
- **(B)** Open addressing with quadratic/perturbation probing
- **(C)** Cuckoo hashing
- **(D)** Robin Hood hashing

79. Since Python 3.6/3.7, how did CPython optimize dictionary memory while guaranteeing insertion order?

- **(A)** Doubly linked lists on entries
- **(B)** Compact array of entries with sparse hash index table
- **(C)** B-tree index
- **(D)** Red-black tree

80. What two dunder methods MUST an object implement to be correctly usable as a dictionary key or in a set?

- **(A)** `__init__` and `__str__`
- **(B)** `__hash__` and `__eq__`
- **(C)** `__cmp__` and `__key__`
- **(D)** `__getitem__` and `__setitem__`

81. If a custom class overrides `__eq__` without defining `__hash__`, what happens to its hashability?

- **(A)** Inherits object.__hash__
- **(B)** `__hash__` is set to None, rendering instances unhashable
- **(C)** Hashes to 0 automatically
- **(D)** Generates random integer hash

82. What is string interning in CPython?

- **(A)** Translating strings to Unicode UTF-32
- **(B)** Reusing memory for identical string literals so `s1 is s2` evaluates True
- **(C)** Encrypting string constants
- **(D)** Compressing strings using LZ4

83. What does `sys.intern('custom_string')` do?

- **(A)** Forces string into CPython's internal string table
- **(B)** Allocates uncollectible memory
- **(C)** Makes string thread-safe
- **(D)** Converts string to ASCII

84. What is the primary memory and performance advantage of `memoryview` when slicing large byte buffers?

- **(A)** Zero-copy slicing without copying underlying buffer memory
- **(B)** Automatic gzip compression
- **(C)** Parallel GPU execution
- **(D)** Converts bytes to floats instantly

85. What is the worst-case time complexity of dictionary key lookup in Python if all keys collide?

(A) O(1)
(B) O(log n)
(C) O(n)
(D) O(n^2)

86. Why are tuples of the same length typically smaller in memory than lists in CPython?

(A) Tuples do not need over-allocation margin or dynamic resizing fields
(B) Tuples compress their elements
(C) Tuples store 16-bit pointers
(D) Tuples cannot store references

87. What optimization does CPython perform for small integer tuples and empty tuples?

(A) Small tuple freelist and single singleton empty tuple `()`
(B) Garbage collector exemption
(C) Compiles to C structs
(D) Converts to C arrays

88. Which built-in module provides a heap queue algorithm (min-heap) over standard lists?

(A) queue
(B) heapq
(C) bisect
(D) trees

89. What is the time complexity of `heapq.heappush(heap, item)` and `heapq.heappop(heap)`?

(A) O(1)
(B) O(log n)
(C) O(n)
(D) O(n log n)

90. What does the `bisect.bisect_left(a, x)` function do?

(A) Sorts list in descending order
(B) Finds the insertion point for x in sorted list a to maintain order
(C) Performs linear search for x
(D) Splits list into two equal halves

91. What happens when attempting to hash an unhashable mutable object like a dictionary?

(A) Returns hash of memory address
(B) Raises TypeError: unhashable type: 'dict'
(C) Returns 0
(D) Hashes string representation

92. What is the purpose of `__slots__` in a custom class regarding memory consumption?

(A) Prevents adding instance methods
(B) Eliminates `__dict__` overhead, storing attributes in fixed C array
(C) Disables garbage collection
(D) Encrypted private attributes

93. What is the average time complexity of `lst.pop(0)` versus `deque.popleft()`?

(A) O(1) vs O(1)
(B) O(n) vs O(1)
(C) O(log n) vs O(1)
(D) O(n) vs O(n)

94. What is the return type and behavior of `d.keys() & {'a', 'b'}` in Python 3?

(A) A set of common keys
(B) A boolean value
(C) TypeError
(D) A list of tuples

95. How does Python handle integer division hashing for numeric types like `hash(1) == hash(1.0)`?

(A) They have different hashes because types differ
(B) Equal numerical values have equal hash values
(C) Raises TypeError
(D) Randomly assigned

96. What is a Trie (Prefix Tree) best suited for when implemented in Python data structures?

(A) Constant-time matrix multiplication
(B) Fast prefix search, autocompletion and dictionary lookup
(C) Sorting floating-point numbers
(D) Simulating analog circuits

97. What is the output of `dict.fromkeys(['a', 'b'], [])` followed by appending to one key's value?

(A) Both keys reflect appended item because both reference the SAME list object
(B) Only key 'a' reflects appended item
(C) Throws TypeError
(D) Creates two distinct lists

98. How can you safely create distinct list values with `dict.fromkeys`?

(A) Use `{k: [] for k in keys}` comprehension
(B) Pass a tuple instead of list
(C) Call `copy()` in fromkeys
(D) Not possible in Python

99. In CPython, what is the structure `PyListObject` composed of?

- **(A)** Doubly linked list of nodes
- **(B)** Array of pointers (`PyObject** ob_item`) plus allocated capacity integer
- **(C)** Contiguous buffer of raw C primitives
- **(D)** B-Tree node array

100. What is the time complexity of the TimSort algorithm used by Python's `list.sort()` and `sorted()`?

- **(A)** O(n^2) worst case
- **(B)** O(n) best case (nearly sorted), O(n log n) worst and average case
- **(C)** O(log n) average
- **(D)** O(n^1.5) worst case

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain the core architectural difference between mutable and immutable data structures in Python with memory diagrams.

Q2. Illustrate the nuances of string slicing in Python, including step parameters, negative indexing, and boundary conditions.

Q3. Compare `list.append()`, `list.extend()`, and list concatenation `+` in terms of memory allocation and behavior.

Q4. Explain the hazards of shallow copying versus deep copying when dealing with nested lists or dictionaries.

Q5. Discuss the internal hashing mechanism of Python dictionaries and explain why dictionary keys must be hashable.

Q6. Describe the difference between Python tuples and lists in terms of memory layout, performance, and semantic intent.

Q7. Demonstrate how to write concise and readable list, dictionary, and set comprehensions with conditional filtering.

Q8. Explain the functionality and use cases of `collections.defaultdict` compared to `dict.setdefault()`.

Q9. Explain the behavior and advantages of `collections.deque` over standard Python lists for FIFO queue operations.

Q10. Describe the mathematical set operations available in Python and demonstrate their syntax with bitwise operators.

Q11. Demonstrate string formatting techniques in Python: % formatting, `str.format()`, and f-strings.

Q12. Explain the `zip()` and `enumerate()` built-in functions with practical engineering examples.

Q13. Explain variable unpacking, nested unpacking, and the star `*` operator in Python sequence assignments.

Q14. Detail the difference between `list.sort()` and `sorted()` in terms of mutation, return values, and parameters.

Q15. Demonstrate the use of the `key` parameter in sorting using lambda functions and `operator.itemgetter`.

Q16. Explain how `bytes` and `bytearray` data structures handle raw binary telecommunication buffers.

Q17. What is a `memoryview` object and how does it prevent unnecessary data copies in high-throughput data processing?

Q18. Describe how to safely remove items from a list or dictionary while iterating over it.

Q19. Explain the `collections.Counter` class with practical examples of frequency counting in digital signals.

Q20. Explain how `itertools.groupby` operates and the strict prerequisite condition required before invoking it.

Q21. Implement an LRU (Least Recently Used) cache using `collections.OrderedDict` with full get and put operations.

Q22. Write a Python implementation of a Trie (Prefix Tree) for efficient string keyword search and prefix matching.

Q23. Derive the time and space complexity of Python's TimSort algorithm used in `list.sort()`.

Q24. Analyze CPython list over-allocation formula and explain why appending to a list is amortized O(1).

Q25. Demonstrate custom class hashability by implementing `__hash__` and `__eq__` for an RF Antenna element.

Q26. Implement a custom circular ring buffer class in Python for realtime sensor streaming.

Q27. Explain Python's dictionary compact layout introduced in CPython 3.6+ and compare it with the legacy hash table.

Q28. Implement a Disjoint Set Union (Union-Find) data structure with path compression and union by rank in Python.

Q29. Explain memory optimization using `__slots__` in Python classes with memory profiling comparisons.

Q30. Implement a Huffman Encoding algorithm using Python dictionaries and `heapq` for binary data compression.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1: [B]
Q2: [B]
Q3: [A]
Q4: [B]
Q5: [B]
Q6: [C]
Q7: [B]
Q8: [B]
Q9: [B]
Q10: [C]
Q11: [C]
Q12: [A]
Q13: [A]
Q14: [C]
Q15: [B]
Q16: [A]
Q17: [C]
Q18: [C]
Q19: [B]
Q20: [B]
Q21: [B]
Q22: [B]
Q23: [B]
Q24: [A]
Q25: [B]
Q26: [D]
Q27: [A]
Q28: [C]
Q29: [C]
Q30: [C]
Q31: [B]
Q32: [B]
Q33: [B]
Q34: [B]
Q35: [B]
Q36: [B]
Q37: [C]
Q38: [A]
Q39: [B]
Q40: [C]
Q41: [B]
Q42: [B]
Q43: [B]
Q44: [C]
Q45: [A]
Q46: [B]
Q47: [B]
Q48: [B]
Q49: [B]
Q50: [B]
Q51: [B]
Q52: [B]
Q53: [B]
Q54: [B]
Q55: [C]
Q56: [A]
Q57: [B]
Q58: [C]
Q59: [B]
Q60: [B]
Q61: [B]
Q62: [B]
Q63: [B]
Q64: [A]
Q65: [B]
Q66: [A]
Q67: [B]
Q68: [B]
Q69: [A]
Q70: [B]
Q71: [A]
Q72: [B]
Q73: [A]
Q74: [B]
Q75: [A]
Q76: [B]
Q77: [B]
Q78: [B]
Q79: [B]
Q80: [B]
Q81: [B]
Q82: [B]
Q83: [A]
Q84: [A]
Q85: [C]
Q86: [A]
Q87: [A]
Q88: [B]
Q89: [B]
Q90: [B]
Q91: [B]
Q92: [B]
Q93: [B]
Q94: [A]
Q95: [B]
Q96: [B]
Q97: [A]
Q98: [A]
Q99: [B]
Q100: [B]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

[illegible]

I

[illegible]

avoiding heap allocation syscalls.

[illegible]

nodes at the block boundaries without moving existing elements.
- Furthermore, `deque(maxlen=K)` provides a hardware-style circular ring buffer where pushing new elements automatically discards the oldest element, ideal for streaming sensor data.

[illegible] with bitwise operators.

Pairs corresponding elements from multiple iterables until the shortest iterable is exhausted:

```python
time_s = [0.0, 0.001, 0.002]
voltage_v = [0.0, 3.25, 4.98]
for t, v in zip(time_s, voltage_v):
print(f't={t}s -> v={v}V')
```

[illegible]

```python
from operator import itemgetter, attrgetter

signals = [
{'channel': 'CH2', 'power_dbm': -12.5},
{'channel': 'CH1', 'power_dbm': -4.2},
{'channel': 'CH3', 'power_dbm': -18.0}
]

# Sort by power ascending using lambda
s_lambda = sorted(signals, key=lambda s: s['power_dbm'])

# Sort by power descending using itemgetter (faster C implementation)
s_getter = sorted(signals, key=itemgetter('power_dbm'), reverse=True)
```

[illegible] data processing?

[illegible]

```
samples.sort(key=lambda s: s[0])
for k, grp in groupby(samples, key=lambda s: s[0]):
print(k, list(grp))
```

[illegible]

```
node = self.root
for char in word:
if char not in node.children:
node.children[char] = TrieNode()
node = node.children[char]
node.is_terminal = True

def starts_with(self, prefix: str) -> bool:
node = self.root
for char in prefix:
if char not in node.children:
return False
node = node.children[char]
return True
```

[illegible]

Suppose we append N elements starting from empty. Reallocations occur only roughly log(N) times. The total number of element copies across all reallocations is bounded by 2N. Dividing the total cost 2N by N appends yields an average cost of <= 2 operations per append, proving O(1) amortized time.

>> Solution to Question 25

Problem Statement: Design and implement a custom circular buffer class in Python for realtime sensor streaming.

Problem Statement: Demonstrate custom class hashability by implementing `__hash__` and `__eq__` for an RF Antenna element.

```
self.size += 1
else:
self.tail = (self.tail + 1) % self.capacity

def get_all(self):
if self.size == 0: return []
if self.head > self.tail:
return self.buffer[self.tail:self.head]
return self.buffer[self.tail:] + self.buffer[:self.head]
```

[illegible]

```
root_j = self.find(j)
if root_i == root_j:
return False
# Union by rank
if self.rank[root_i] < self.rank[root_j]:
self.parent[root_i] = root_j
elif self.rank[root_i] > self.rank[root_j]:
self.parent[root_j] = root_i
else:
self.parent[root_j] = root_i
self.rank[root_i] += 1
return True
```

[illegible]

```
n1 = heapq.heappop(heap)
n2 = heapq.heappop(heap)
merged = HuffmanNode(None, n1.freq + n2.freq)
merged.left, merged.right = n1, n2
heapq.heappush(heap, merged)

codes = {}
def generate(node, prefix=''):
if node.char is not None:
codes[node.char] = prefix
return
generate(node.left, prefix + '0')
generate(node.right, prefix + '1')
generate(heap[0])
return codes
```
